Southern New Hampshire University

CS 300 6-2 Project One

Karina Valadez

Runtime Analysis Charts

Vector Runtime Analysis Chart

Code	Line Cost	# Times Executes	Total Cost
PRINT "Enter file name:"	1	1	1
READ filename	1	1	1
OPEN filename	1	1	1
IF filename does not open	1	1	1
WHILE filename has another line	1	n	n
READ line from filename	1	n	n
SEPARATE line into tokens	1	n	n
IF number of tokens is less than 2	1	n	n
CREATE newCourse AS Course	1	n	n
SET newCourse.courseNumber to tokens[0]	1	n	n
SET newCourse.courseName to tokens[1]	1	n	n
FOR EACH token starting after the first two values	1	n	n
ADD token to newCourse.prerequisites	1	n	n
ADD newCourse to courses	1	n	n
CLOSE filename	1	1	1
FOR EACH course in courses	1	n	n
FOR EACH prerequisite in course.prerequisites	1	n	n
SET prerequisiteFound to FALSE	1	n	n
FOR EACH otherCourse in courses	1	n^2	n^2
IF otherCourse.courseNumber equals prerequisite	1	n^2	n^2
SET prerequisiteFound to TRUE	1	n	n
IF prerequisiteFound equals FALSE	1	n	n
RETURN TRUE	1	1	1
		Total Cost	$2n^2 + 15n + 6$
		Runtime	$O(n^2)$

Southern New Hampshire University

Hash Table Runtime Analysis Chart

Code	Line Cost	# Times Executes	Total Cost
PRINT "Enter file name:"	1	1	1
READ filename	1	1	1
OPEN filename	1	1	1
IF filename does not open	1	1	1
WHILE filename has another line	1	n	n
READ line from filename	1	n	n
SEPARATE line into parameters	1	n	n
IF number of parameters is less than 2	1	n	n
CREATE newCourse AS Course	1	n	n
SET newCourse.courseNumber to parameters[0]	1	n	n
SET newCourse.courseName to parameters[1]	1	n	n
FOR EACH parameter starting after the first two values	1	n	n
ADD parameter to newCourse.prerequisites	1	n	n
INSERT newCourse into coursesHashTable USING newCourse.courseNumber as the key	O(n)	n	$O(n^2)$
CLOSE filename	1	1	1
FOR EACH course in coursesHashTable	1	n	n
FOR EACH prerequisite in course.prerequisites	1	n	n
IF prerequisite does not exist in coursesHashTable	O(n)	n	$O(n^2)$
RETURN TRUE	1	1	1
		Total Cost	$O(n^2)$
		Runtime	$O(n^2)$

Southern New Hampshire University

Binary Search Tree Runtime Analysis Chart

Code	Line Cost	# Times Executes	Total Cost
DEFINE courseData AS LIST	1	1	1
OPEN courseFile	1	1	1
IF courseFile does not open	1	1	1
WHILE courseFile has another line	1	n	n
READ line	1	n	n
SEPARATE line into tokens	1	n	n
IF number of tokens is less than 2	1	n	n
ADD tokens to courseData	1	n	n
CLOSE courseFile	1	1	1
FOR EACH courseLine in courseData	1	n	n
FOR EACH prerequisite in courseLine starting after the first two values	1	n	n
SET prerequisiteFound to FALSE	1	n	n
FOR EACH otherCourseLine in courseData	1	n^2	n^2
IF prerequisite equals otherCourseLine[0]	1	n^2	n^2
SET prerequisiteFound to TRUE	1	n	n
IF prerequisiteFound equals FALSE	1	n	n
FOR EACH courseLine in courseData	1	n	n
CREATE newCourse AS Course	1	n	n
SET newCourse.courseNumber to courseLine[0]	1	n	n
SET newCourse.courseName to courseLine[1]	1	n	n
FOR EACH value in courseLine starting after the first two values	1	n	n
ADD value to newCourse.prerequisites	1	n	n
SET courseTree.root to insertCourse(courseTree.root, newCourse)	O(n)	n	$O(n^2)$
RETURN TRUE	1	1	1
		Total Cost	$O(n^2)$
		Runtime	$O(n^2)$

Southern New Hampshire University

Structure Advantages and Disadvantages Evaluations

Vector Data Structure

The Vector's data structure has the worst-case runtime $O(n^2)$ because the program has to check each one of the course's prerequisites using nested loops. One of the advantages of a vector is that is easy to implement, allows direct access to the elements and uses little memory. One disadvantage is that to find a specific course it can take longer because the program may have to search through the entire list. Another disadvantage is that the courses must be sorted before they are printed in alphanumeric order. So even though a vector is easy to use it is less efficient the more courses there are.

Hash Table Structure

The Hash Table data structure has the worst-case runtime $O(n^2)$ because multiple courses can end up in the same location making it longer to store or to find them. Another disadvantage is that a hash table does not automatically keep courses in order, so the program has to sort them before printing the full list. Hash tables also require a lot more memory for storing and organizing. One advantage is that the average insertion and lookup times are O(1), making it very efficient for finding a course directly using the course number.

Binary Search Tree Structure

The Binary Search Tree has the worst-case runtime $O(n^2)$ because the pseudocode uses nested loops to validate prerequisites and inserting courses into an unbalanced tree makes the

runtime slower. One advantage is that the Binary Search Tree organizes course numbers according to their values, this allows the program to print them in alphanumeric order without sorting them separately. It can also find the courses much faster when the tree is balanced.

Recommendation

My recommendation is to use the Binary Search Tree for the ABCU advising program. Even though the analysis shows that all three data structures have a worst-case runtime of $O(n^2)$, one of the main requirements of the ABCU advising program is to print courses in alphanumeric order. The Binary Search Tree supports this requirement through an in-order traversal, which allows the program to print courses in the correct order without needing to sort them separately. Although the other data structures have their own advantages, I believe the Binary Search Tree is the most suitable choice because it better meets the main requirements of the ABCU advising program.

Citations

DSANotes. (n.d.). Frequency count method. Retrieved October 9, 2026, from https://dsanotes.com/learn/frequency-count

GeeksforGeeks. (n.d.). Advantages of BST over hash table. Retrieved October 9, 2026, from https://www.geeksforgeeks.org/dsa/advantages-of-bst-over-hash-table/